



EPIGRAM: Practical Garbled RAM

David Heath^{1(✉)}, Vladimir Kolesnikov¹, and Rafail Ostrovsky²

¹ Georgia Tech, Atlanta, USA

{heath.davidanthony,kolesnikov}@gatech.edu

² UCLA, Los Angeles, USA

rafail@cs.ucla.edu

Abstract. Garbled RAM (GRAM) is a powerful technique introduced by Lu and Ostrovsky that equips Garbled Circuit (GC) with a sublinear cost RAM without adding rounds of interaction. While multiple GRAM constructions are known, none are suitable for practice, due to costs that have high constants and poor scaling.

We present the first GRAM suitable for practice. For computational security parameter κ and for a size- n RAM that stores blocks of size $w = \Omega(\log^2 n)$ bits, our GRAM incurs amortized $O(w \cdot \log^2 n \cdot \kappa)$ communication and computation per access. We evaluate the concrete cost of our GRAM; our approach outperforms trivial linear-scan-based RAM for as few as 512 128-bit elements.

Keywords: MPC · Garbled circuits · Oblivious RAM · Garbled RAM

1 Introduction

Secure multiparty computation (MPC) allows mutually untrusting parties to compute functions of their combined inputs while revealing nothing but the outputs. MPC protocols traditionally consider functions encoded as circuits. While this does not limit expressivity, it does limit efficiency: many interesting computations are best expressed as RAM programs, not as circuits, and the reduction from RAM programs to circuits is expensive.

Fortunately, we can combine MPC with oblivious RAM (ORAM). ORAM is a technology that allows a client to outsource an encrypted database to a server; the client can then access the database while both (1) incurring only sublinear overhead and (2) hiding the access pattern from the server. By running an ORAM client inside MPC, we can augment circuits with random access memory. This powerful combination allows us to run RAM programs inside MPC.

Garbled Circuit (GC) is a foundational and powerful MPC technique that allows two parties to achieve secure computation while consuming only constant rounds of interaction. One party, the GC generator G , “encrypts” the circuit and sends it to the other party, the GC evaluator E . E is given an encryption of each party’s input and steps through the circuit gate-by-gate under encryption. At each gate, E propagates encryptions of input wire values to encryptions of

output wire values. Once E finishes, E and G can jointly decrypt the output wire values, revealing the circuit output.

It is natural to consider adding RAM to GC while preserving GC’s constant rounds. However, the constant round requirement means that adding RAM to GC is seemingly more difficult than adding RAM to interactive protocols. Nevertheless, it is possible to run an ORAM client inside the GC and to let E play an ORAM server. This technique is called Garbled RAM (GRAM) [LO13].

While GRAM constructions are known [LO13, GHL+14, GLOS15, GLO15], none are suitable for practice: existing constructions simply cost too much. All existing GRAMs suffer from at least two of the following problems:

- **Use of non-black-box cryptography.** [LO13] showed that GRAM can be achieved by evaluating a PRF *inside GC* in a non-black-box way. Unfortunately, this non-black-box cryptography is extremely expensive, and on each access the construction must evaluate the PRF *repeatedly*. [LO13] requires a circular-security assumption on GC and PRFs. Follow-up works removed this circularity by replacing the PRF with even more expensive non-black-box techniques [GHL+14, GLOS15].
- **Factor- κ blowup.** Let κ denote the computational security parameter. In practical GC, we generally assume that we will incur factor κ overhead due to the need to represent each bit as a length- κ *encoding* (i.e. a GC label). However, existing GRAMs suffer from yet another factor κ . This overhead follows from the need to represent GC labels (which have length κ) *inside the GC* such that we can manipulate them with Boolean operations. The GC labels that encode a GC label together have length κ^2 . In practice, where we generally use $\kappa = 128$, this overhead is intolerable.
- **High factor scaling.** Existing GRAMs operate as follows. First, they give an array construction that leaks access patterns to E . This leaky array already has high cost. Then, they compile this array access into GRAM using off-the-shelf ORAM. This compilation is problematic: off-the-shelf ORAMs require that, on each access, E access the leaky array a polylogarithmic (or more) number of times. Thus, existing GRAMs incur *multiplicative* overhead from the composition of the leaky array with the ORAM construction.

Prior GRAM works do not attempt to calculate their concrete or even asymptotic cost, other than to claim cost sublinear or polylogarithmic in n . In the full version of this paper, we (favorably to prior work) estimate their cost: for a GRAM that stores 128-bit blocks, the best prior GRAM breaks even with trivial linear-scan based GRAM when the RAM size reaches $\approx 2^{20}$ elements. As noted, our analysis discounts many potentially expensive steps of prior constructions, giving an estimate favorable to them. In particular, this conservative estimate indicates that by the time it is worthwhile to use existing GRAM, each and every access requires a 4GB GC.

1.1 Contribution

We present the first practical garbled RAM. Our GRAM, which we call EPIGRAM, uses only $O(w \cdot \log^2 n \cdot \kappa)$ computation and communication per access. EPIGRAM circumvents all three of the above problems:

- **No use of non-black-box cryptography.** Our approach routes array elements using novel, yet simple, techniques. These techniques are light-weight, and non-black-box cryptography is not required.
- **No factor- κ blowup.** While we, like previous GRAMs, represent GC labels inside the GC itself, we give a novel generalization of existing GC gates that eliminates the additional factor κ overhead.
- **Low polylogarithmic scaling.** Like previous GRAMs, we present a leaky construction that reveals access patterns to E . However, we do not compile this into GRAM using off-the-shelf ORAM. Instead, we construct a custom ORAM designed with GC in mind. Our GRAM minimizes use of our leaky construction. The result is a highly efficient technique.

In the remainder of this paper we:

- Informally and formally describe the first practical GRAM. For an array with n elements each of size w such that $w = \Omega(\log^2 n)$, the construction incurs amortized $O(w \cdot \log^2 n \cdot \kappa)$ communication and computation per access.
- Prove our GRAM secure by incorporating it in a *garbling scheme* [BHR12]. Our scheme handles arbitrary computations consisting of AND gates, XOR gates, and array accesses. Our scheme is secure under a typical GC assumption: a circular correlation robust hash function [CKKZ12].
- Analyze EPIGRAM’s concrete cost. Our analysis shows that EPIGRAM outperforms trivial linear-scan based RAM for as few as 512 128-bit elements.

2 Technical Overview

In this section, we explain our construction informally but with sufficient detail to understand our approach. This overview covers four topics:

- First, we explain a problem central to GRAM: *language translation*.
- Second, we informally explain our *lazy permutation network*, which is a construction that efficiently solves the language translation problem.
- Third, as a stepping stone to our full construction, we explain how to construct *leaky* arrays from the lazy permutation network. This informal construction securely implements an array with the caveat that we let E learn the array access pattern.
- Fourth, we upgrade the leaky array to full-fledged GRAM: the presented construction hides the access pattern from E .

2.1 The Language Translation Problem

For each GC wire x_i the evaluator E holds one of two κ -bit strings: either X_i , which encodes a logical zero, or $X_i \oplus \Delta$, which encodes one. Meanwhile, G holds each such X_i and the global secret Δ . We refer to the wire-specific value X_i as the *language* of that wire, and to the pair $\langle X_i, X_i \oplus \Delta \rangle$ jointly held by G and E as the *GC encoding*, or the *garbling*, of x_i . We present this notation formally in

Sect. 4.4. To produce a garbled gate that takes as input a particular wire value x_i , G must know the corresponding language X_i . Normally this is not a problem: the structure of the circuit is decided statically, and G can easily track which languages go to which gates.

However, consider representing an array as a collection of such garbled labels. That is, there are n values x_i where E holds $X_i \oplus x_i \Delta$. Suppose that at runtime the GC requests access to a particular index α . We could use a static circuit to select x_α , but this would require an expensive linear-cost circuit. A different method is required to achieve the desired sublinear access costs.

Instead, suppose we disclose α to E in cleartext – we later add mechanisms that hide RAM indices from E . Since she knows α , E can jump directly to the α th wire and retrieve the value $X_\alpha \oplus x_\alpha \Delta$. Recall, to use a wire as input to a gate, G and E must agree on that wire’s language. Unfortunately, it is *not possible* for G to predict the language X_α : α is computed during GC evaluation and, due to the constant round requirement, E cannot send messages to G .

Therefore, we instead allow G to select a fresh uniform language Y . If we can convey to E the value $Y \oplus x_\alpha \Delta$, then G will be able to garble gates that take the accessed RAM value as input, and we can successfully continue the computation.

Thus, our new goal is to *translate* the language X_α to the language Y . Mechanically, this translation involves giving to E the value $X_\alpha \oplus Y$. Given this, E simply XORs the translation value with her label and obtains $Y \oplus x_\alpha \Delta$. Keeping the circuit metaphor, providing such translation values to E allows her to take two wires – the wire out of the RAM and the wire into the next gate – and to solder these wires together *at runtime*. However, the problem of efficiently conveying these translation values remains.

In the full version of this paper, we discuss natural attempts at solving the language translation problem. Translation can be achieved by a linear-sized gadget (suggesting dynamic conversion is possible), or by a non-black box PRF [LO13] (suggesting the ability to manipulate languages inside the GC). Our *lazy permutation network* (discussed next) achieves dynamic language translation more cheaply, but its underpinnings are the same: the network carefully manipulates languages inside the GC.

2.2 Lazy Permutations

Recall that our current goal is to translate GC languages. Suppose that the GC issues n accesses over its runtime. Further suppose that the GC accesses a *distinct location* on each access – in the end we reduce general RAM to a memory with this restriction. To handle the n accesses, we wish to convey to E n translation values $X_i \oplus Y_j$ where Y_j is G ’s selected language for the j th access.

What we need then is essentially a permutation on n elements that routes between RAM locations (with language X_i) and accesses (with language Y_j). However, a simple permutation network will not suffice, since at the time of RAM access j , the location of each subsequent access will, in general, not yet be known. Therefore, we need a *lazy* permutation whereby we can decide and apply the routing of the permutation one input at a time. We remind the reader

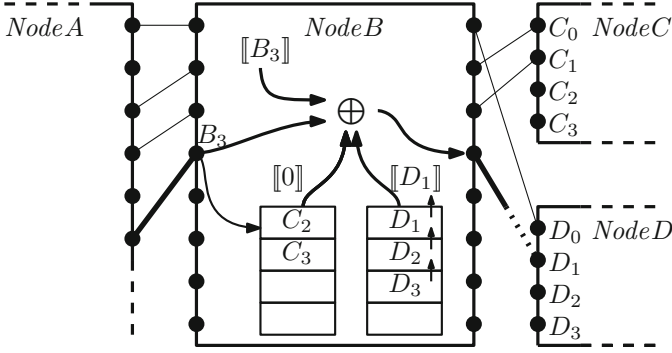


Fig. 1. An internal node of our lazy permutation network, realizing a “garbled switch”. We depict the fourth access to this switch. The encoded input uses language B_3 . The first encoded input bit is a flag that indicates to proceed left or right. Our objective is to forward the remaining input to either the left or right node. Each node stores two oblivious stacks that hold encodings of the unused languages of the two children. We conditionally pop both stacks. In this case, the left stack is unchanged whereas the right stack yields D_1 , the next language for the target child. Due to the pop, the remaining elements in the right stack move up one slot. By XORing these values with an encoding of the input language, then opening the resulting value to E , we convert the message to the language of the target child, allowing E to solder a wire to the child.

that we assume that E knows each value α . I.e., we need only achieve a lazy permutation where E learns the permutation.

Given this problem, it may now be believable that algorithms and data structures exist such that the total cost is $O(\kappa \cdot n \cdot \text{polylog}(n))$, and hence only amortized $O(\kappa \cdot \text{polylog}(n))$ per access. Indeed we present such a construction. However, our solution requires that we apply this lazy permutation *to the GC languages themselves*, not to bits stored in the RAM. Thus, we need a logic in which we can encode GC languages: E must obviously and authentically manipulate GC languages. GC gives us these properties, so we can encode languages bit-by-bit inside the GC. I.e., for a language of length w , we would add w GC wires, each of which would hold a single bit of the language.

Unfortunately, this bit-by-bit encoding of the languages leads to a highly objectionable factor κ blowup in the size of the GC: the encoding of a length- w language has length $w \cdot \kappa$. We later show that the factor κ blowup is unnecessary. Under particular conditions, existing GC gates can be generalized such that we can represent a length- w language using an encoding of only length w . These special and highly efficient GC gates suffice to build the gadgetry we need. We formalize the needed gate in Sect. 5.1.

The ability to encode languages inside the GC is powerful. Notice that since we can dynamically solder GC wires, and since wires can hold languages needed to solder other wires, we can arrange for E to repeatedly and dynamically lay down new wiring in nearly arbitrary ways.

With this high level intuition, we now informally describe our lazy permutation network. Let n be a power of two. Our objective is to route between the languages of n array accesses and the languages of n array elements.

G first lays out a full binary tree with n leaves. Each node in this tree is a GC with static structure. However, the inputs and outputs to these circuits are loose wires, ready to be soldered at runtime by E . At runtime, seeking to read array element x_α with language X_α into a wire with language Y , E begins at the root of the tree, which holds a GC encoding of the target language Y . (Note, G knows the target language Y of the j -th access, and can accordingly program the tree root.) Based on the GC encoding of the first bit of α , E is able to dynamically decrypt a translation value to either the left or the right child node. Now, E can solder wires to this child, allowing her to send to the child circuit both the encoding of Y and the remaining bits of α . E repeatedly applies this strategy until she reaches the α th leaf node. This leaf node is a special circuit that computes $\mathcal{C}(x) = x \oplus X_\alpha$ and then reveals the output to E .¹ Since we have pushed the encoding of Y all the way to this leaf, E obtains $Y \oplus X_\alpha$, the translation value that she needs to read x_α .

In yet more detail, each internal node on level k of the tree, which we informally call a *garbled switch*, is a static circuit with $2^{\log n - k}$ loose sets of input wires. Each node maintains two *oblivious stacks* [ZE13]. The first stack stores encodings of the languages for the $2^{\log n - k - 1}$ loose input wires of the left child, and the second stack similarly stores languages for the right child (see Fig. 1). On the j -th access and seeking to compute $Y_j \oplus X_\alpha$, E dynamically traverses the tree to leaf α (recall, we assume E knows α in cleartext), forwarding an encoding of Y_j all the way to the α th leaf. At each internal node, she uses a bit of the encoding of α to conditionally pop the two stacks, yielding an encoding of the language of the correct child. The static circuit uses this encoding to compute a translation value to the appropriate child.

By repeatedly routing inputs over the course of n accesses, we achieve a lazy permutation. Crucially, the routing between nodes is not decided until runtime.

This construction is affordable. Essentially the only cost comes from the oblivious stacks. For a stack that stores languages of length w , each pop costs only $O(w \cdot \log n)$ communication and computation (Sect. 5.2). Thus, the full lazy permutation costs only $O(w \cdot n \cdot \log^2 n)$ communication, which amortizes to sublinear cost per access. We describe our lazy permutation network in full formal detail in Sect. 5.3.

Our lazy permutation networks route the language of each RAM slot to the access where it is needed, albeit in a setting where E views the routing in cleartext. Crucially, the lazy permutation network avoids factor κ additional overhead that is common in GRAM approaches. To construct a secure GRAM, we build on this primitive and hide the RAM access pattern.

¹ Our actual leaf circuit is more detailed. See Sects. 2.4 and 5.3.

2.3 Pattern-Leaking (Leaky) Arrays

As a stepping stone to full GRAM, we informally present an intermediate array which leaks access patterns. For brevity, we refer to it as *leaky* array. This construction handles arbitrary array accesses in a setting where E is allowed to learn the access pattern. We demonstrate a reduction from this problem to our lazy permutation network.

We never *formally* present the resulting construction. Rather, we explain the construction now for expository reasons: we decouple our explanation of *correctness* from our explanation of *obliviousness*. I.e., this section builds a correct GRAM that leaks the access pattern to E . The ideas for this leaky construction carry to our secure GRAM (Sect. 2.4).

Suppose the GC wishes to read index α . Recall that our lazy permutation network is a mechanism that can help translate GC languages: E can dynamically look up an encoding of the language X_α . However, because the network implements a *permutation*, it alone does not solve our problem: an array should allow multiple accesses to the same index, but the permutation can route each index to *only one* access. To complete the reduction, more machinery is needed.

To start, we simplify the problem: consider an array that handles at most n accesses. We describe an array that works in this restricted setting and later upgrade it to handle arbitrary numbers of accesses.

Logical Indices $\rightarrow$ One-Time Indices. The key idea is to introduce a level of indirection. While the GC issues queries via logical indices α , our array stores its content according to a different indexing system: the content for each logical index α is stored at a particular *one-time index* p . As the name suggests, each one-time index may be written to and read at most once. This limitation ensures compatibility with a lazy permutation: since each one-time index is read only once, a permutation suffices to describe the read pattern. We remark that this reduction from general purpose RAM to a permuted read order was inspired by prior work on efficient RAM for Zero Knowledge [HK21].

Each one-time index can be read only once, yet each logical index can be read multiple times. Thus, over the course of n accesses, a given logical index might correspond to *multiple* one-time indices.

Neither party can *a priori* know the mapping between logical indices and one-time indices. However, to complete an access the GC must compute the relevant one-time index. Thus, we implement the mapping as a recursively instantiated *index map*.² The index map is itself a leaky array where each index α holds the corresponding one-time index p . We are careful that the index map is strictly smaller than the array itself, so the recursion terminates; when the next needed index map is small enough, we instantiate it via simple linear scans.

A leaky array with n elements each of size w and that handles at most n accesses is built from three pieces:

1. A block of $2n$ GC encodings each of size w called the one-time array. We index into the one-time array using one-time indices.

² Recursive index/position maps are typical in ORAM constructions, see e.g. [SvS+13].

2. A size- $2n$ lazy permutation $\tilde{\pi}$ where each leaf i stores the language for one-time array slot i .
3. The recursively instantiated index map.

Let $\{x_i\}$ denote the GC encoding of bitstring x_i where G holds X_i and E holds $X_i \oplus x_i \Delta$ (see also Sect. 4.4). Suppose the parties start with a collection of n encodings $\{x_0\}, \dots, \{x_{n-1}\}$ which they would like to use as the array content. The parties begin by sequentially storing each value $\{x_i\}$ in the corresponding one-time index i . The initial mapping from logical indices to one-time indices is thus statically decided: each logical index i maps to one-time index i . The parties recursively instantiate the index map with content $\{0\}, \dots, \{n-1\}$.

When the GC performs its j -th access to logical index $\{\alpha\}$, we perform the following steps:

1. The parties recursively query the index map using input $\{\alpha\}$. The result is a one-time index $\{p\}$. The parties simultaneously write back into the index map $\{n+j\}$, indicating that α will next correspond to one-time index $n+j$.
2. The GC reveals p to E in cleartext. This allows E to use the lazy permutation network $\tilde{\pi}$ to find a translation value for the p th slot of the one-time array.
3. E jumps to the p th slot of the array and translates its language, soldering the value to the GC and completing the read. Note that the GC may need to access index α again, so the parties perform the next step:
4. The parties write back to the $(n+j)$ -th slot of the one-time array. If the access is a read, they write back the just-read value. Otherwise, they write the written value.

In this way, the parties can efficiently handle n accesses to a leaky array.

Handling More than n Accesses. If the parties need more than n accesses, a reset step is needed. Notice that after n accesses, we have written to each of the $2n$ one-time indices (n during initialization and one per access), but we have only read from n one-time indices. Further notice that on an access to index α , we write back a new one-time index for α ; hence, it must be the case that the n remaining unread one-time array slots hold the current array content.

Going beyond n accesses is simple. First, we one-by-one read the n array values in the sequential *logical* order (i.e. with $\alpha = 0, 1, \dots, n-1$), flushing the array content into a block $\{x_0\}, \dots, \{x_{n-1}\}$. Second, we initialize a new leaky array data structure, using the flushed block as its initial content. This new data structure can handle n more accesses. By repeating this process every n accesses, we can handle arbitrary numbers of accesses.

Summarizing the Leaky Array. Thus, we can construct an efficient garbled array, which leaks access patterns. Each access to the leaky array costs amortized $O(w \cdot \log^2 n \cdot \kappa)$ bits of communication, due to the lazy permutation network. We emphasize the key ideas that carry over to our secure GRAM:

- We store the array data according to *one-time indices*, not according to logical indices. This ensures compatibility with our lazy permutation network.

- We recursively instantiate an *index map* that stores the mapping from logical indices to one-time indices.
- We store the GC languages of the underlying data structure in a lazy permutation network such that E can dynamically access slots.
- Every n accesses, we *flush* the current array and instantiate a fresh one.

2.4 Garbled RAM

In Sect. 2.3 we demonstrated that we can reduce random access arrays to our lazy permutation network, so long as E is allowed to learn the access pattern. In this section we strengthen that construction by hiding the access patterns, therefore achieving secure GRAM.

Note that this strengthening is clearly possible, because we can simply employ off-the-shelf ORAM. In ORAM, the server learns a physical access pattern, but the ORAM protocol ensures that these physical accesses together convey no information about the logical access pattern. Thus, we can use our leaky array to implement physical ORAM storage, implement the ORAM client inside the GC, and the problem is solved.

We are not content with this solution. The problem is that our leaky array already consumes $O(\log^2 n)$ overhead, due to lazy permutations. In ORAM, each logical access is instantiated by at least a logarithmic number of physical reads/writes. Thus, compiling our leaky array with off-the-shelf ORAM incurs *at least* an additional $O(\log n)$ *multiplicative factor*. In short, this off-the-shelf composition is expensive.

We instead directly improve the leaky array construction (Sect. 2.3) and remove its leakage. This modification incurs only additive overhead, so our GRAM has the same asymptotic cost as the leaky array: $O(w \cdot \log^2 n \cdot \kappa)$ bits per access.

The key idea of our full GRAM is as follows: In regular ORAM, we assume that the client is significantly weaker than the server. In our case, too, the GC – which plays the client – *is* much weaker than E – who plays the server. However, we have a distinct advantage: the GC generator G can act as a powerful advisor to the GC, directly informing most of its decisions.

More concretely, our GRAM carefully arranges that the locations of almost all of the physical³ reads and writes are decided *statically* and are independent of the logical access pattern. Thus, G can *a priori* track the static schedule and prepare for each of the static accesses. Our GRAM incurs $O(\log^2 n)$ physical reads/writes per logical access. However, only a *constant number*⁴ of these reads cannot be predicted by G , as we will soon show.

Each physical read/write requires that G and E agree on the GC language of the accessed element. For each statically decided read/write, this agreement

³ I.e. reads and writes to the lowest level underlying data structure, where access patterns are visible to E .

⁴ To be pedantic, if we account for recursively instantiated index maps, each map incurs this constant number of unpredictable reads, so there are total a logarithmic number of unpredictable reads.

is reached trivially. Therefore, we only need our lazy permutation network for reads that G cannot predict. There are only a constant number of these, so we only need a constant number of calls to the lazy permutation network.

Upgrading the Leaky Array. We now informally describe our GRAM. Our description is made by comparison to the leaky array described in Sect. 2.3.

In the leaky array, we stored all $2n$ one-time indices in a single block. In our GRAM, we instead store the $2n$ one-time indices across $O(\log n)$ levels of exponentially increasing size: each level i holds 2^{i+1} elements, though some levels are vacant. As we will describe later, data items are written to the smallest level and then slowly move from small levels to large levels. Each populated level of the GRAM holds 2^i one-time-indexed data items and 2^i *dummies*. Dummies are merely encodings of zero. Each level of the GRAM is stored shuffled. The order of items on each level is unknown to E but, crucially, *is known to G* . This means that at all times G knows which one-time index is stored where and knows which elements are dummies.

In the leaky array, E was pointed directly to the appropriate one-time index. In our GRAM, we need to hide the identity of the level that holds the appropriate index. Otherwise, since elements slowly move to larger levels, E will learn an approximation of the time at which the accessed element was written. Hence we arrange that E will read from *each* level on each access. However, all except one of these accesses will be to a dummy, and the indices of the accessed dummies are *statically scheduled by G* . More precisely, G *a priori* chooses one dummy on *each* populated level and enters their addresses as input to the GC. The GC then conditionally replaces one dummy address by the real address, then reveals each address to E . (Note that G *does not know* which dummy goes unaccessed – we discuss this later.)

In the leaky array and when accessing logical index α , we used the index map to find corresponding one-time index p . p was then revealed to E . In our GRAM, it is not secure for E to learn one-time indices corresponding to accesses. Thus, we introduce a new uniform permutation π of size $2n$ that is held by G and secret from E . Our index map now maps each index $\{\alpha\}$ to the corresponding *permuted* one-time index $\{\pi(p)\}$. We can safely reveal $\pi(p)$ to E – the sequence of such revelations is indistinguishable from a uniform permutation.

In the leaky array, we used the lazy permutation network $\tilde{\pi}$ to map each one-time index p to a corresponding GC language. Here, we need two changes:

1. Instead of routing p to the metadata corresponding to p , we instead route $\pi(p)$ to the metadata corresponding to p . G can arrange for this by simply initializing the content of the lazy permutation in permuted order.
2. We slowly move one-time indexed array elements from small levels to large levels (we have not yet presented how this works). Thus, each one-time index no longer corresponds to a single GC language. Instead, each one-time index now corresponds to a *collection* of physical addresses. Moreover, each time we move a one-time index to a new physical address, it is crucial to security that we encode the data with a different GC language. Fortunately, we ensure

that G knows the entire history of each one-time index. Thus, he can garble a circuit that takes as input the number of accesses so far and outputs the *current* physical address and GC language.

We place these per-one-time-index circuits at the leaves of a lazy permutation network.

Remark 1 (Indices). Our GRAM features three kinds of indices:

- *Logical indices* α refer to simple array indices. The purpose of the GRAM is to map logical indices to values.
- Each time we access a logical index, we write back a corresponding value to a fresh *one-time index* p . Thus, each logical index may correspond to multiple one-time indices. The mapping from logical indices to one-time indices is implemented by the recursively instantiated *index map*.
- One-time indices are not stored sequentially, but rather are stored permuted such that we hide access patterns from E . A *physical address* $@$ refers to the place where a one-time index p is currently held. Because we repeatedly move and permute one-time indices, each one-time index corresponds to multiple physical addresses. The mapping from one-time indices to physical addresses is known to G and is stored in a lazy permutation network.

In the leaky array and on access j , we write back an element to one-time index $n + j$. In our GRAM, we similarly perform this write. We initially store this one-time index in the smallest level. Additionally, the parties store a fresh dummy in the smallest level. After each write, the parties permute a subset of the levels of RAM using a traditional permutation network. The schedule of permutations – see next – is carefully chosen such that the access pattern is hidden but cost is low. Over the course of n accesses, the n permutations together consume only $O(n \cdot \log^2 n)$ overhead.

The Permutation Schedule. Recall that we arrange the RAM content into $O(\log n)$ levels of exponentially increasing size. After each access, G applies a permutation to a subset of these levels. These permutations prevent E from learning the access pattern.

Recall that on each access, E is instructed to read from each populated level. All except one of these reads is to a dummy. Further recall that after being accessed once, a one-time index is never used again. Thus, it is important that each dummy is similarly accessed at most once. Otherwise, E will notice that doubly-accessed addresses must hold dummies.

Since we store only 2^i dummies on level i , level i can only support 2^i accesses: after 2^i accesses it is plausible that all dummies have been exhausted. To continue processing, G therefore re-permutes the level, mixing the dummies and real elements such that the dummies can be safely reused. More precisely, on access j we collect those levels i such that 2^i divides j . Let k denote the largest such i . We concatenate each level $i \leq k$ together into a block of size 2^{k+1} and permute its contents into level $k + 1$ (this level is guaranteed to be vacant). This leaves

each level $i \leq k$ vacant and ready for new data to flow up. Now that the data has been permuted, it is safe to once again use the shuffled dummies, since they are shuffled and each is given a new GC language.

As a security argument, consider E 's view of a particular level i over all 2^i accesses between permutations. Each such access could be to a dummy or to a real element, but these elements are uniformly shuffled. Hence, E 's view can be simulated by uniformly sampling, without replacement, a sequence of 2^i indices.

Remark 2 (Permutations). Our RAM features three kinds of permutations:

- $\tilde{\pi}$ is a *lazy* permutation whose routing is revealed to E over the course of n accesses. The lazy permutation allows E to efficiently look up the physical address and language for the target one-time index.
- π is a uniform permutation chosen by G whose sole purpose is to ensure that $\tilde{\pi}$ does not leak one-time indices to E . Let π' denote the *actual* routing from RAM accesses to one-time indices. E does not learn π' , but rather learns $\tilde{\pi} = \pi' \circ \pi$. Since π is uniform, $\tilde{\pi}$ is also uniform.
- $\pi_0, \dots, \pi_{n-1}$ is a sequence of permutations chosen by G and applied to levels of GRAM. These ensure that the physical access pattern leaks nothing to E .

Accounting for the Last Dummy per Access. One small detail remains. Recall that on each access, G statically chooses a dummy on each of the $O(\log n)$ levels. E will be pointed to each of these dummies, save one: E will not read the dummy on the same level as the real element. The identity of the real element is dynamically chosen, so G cannot know which dummy is not read. The parties must somehow account for the GC language of the unread dummy to allow E to proceed with evaluation. (We expand on this need in a moment.)

This accounting is easily handled by a simple circuit $\mathcal{C}_{hide}$. $\mathcal{C}_{hide}$ takes as input an encoding of the real physical address and outputs an encoding of the language of the unaccessed dummy.

We now provide more detail (which can be skipped at the first reading) explaining why E must recover an encoding of the language of the unaccessed dummy. Suppose the real element is on level j . G selects $O(\log n)$ dummy languages D_i for this access, and E reads one label in each language $D_{i \neq j}$, and reads the real value. To proceed, G and E must obtain the real value in some agreed language, and this language must depend on all languages D_i (since G cannot know which dummy was not read). Therefore, D_j must be obtained and used by E as well. In even more detail, in the mind of G , the “output” language includes the languages D_i XORed together; to match this, in addition to XORing all labels she already obtained, E XORs in the encoding of the missing dummy language. The validity of this step relies heavily on Free XOR [KS08].

The High Level Procedure. To conclude our overview, we enumerate the steps of the RAM. Consider an arbitrary access to logical index α .

1. E first looks up α 's current one-time index p by consulting the index map. The index map returns an encoding of $\pi(p)$ where π is a uniform permutation that hides one-time indices from E .
2. The GC reveals $\pi(p)$ to E in cleartext such that she can route the lazy permutation $\tilde{\pi}$. E uses $\tilde{\pi}$ to route the current RAM time to a leaf circuit that computes encodings of the appropriate physical address $@$ and GC language. Let ℓ denote the RAM level that holds address $@$.
3. A per-access circuit C_{hide} is used to compute (1) encodings of physical addresses of dummies on each populated level $i \neq \ell$ and (2) the GC language of the dummy that *would* have been accessed on level ℓ , had the real element been on some other level.
4. The GC reveals addresses to E and E reads each address. E XORs the results together. (Recall, dummies are garblings of zero.) Each read value is a GC label with a distinct language. To continue, G and E must agree on the language of the resulting GC label. G can trivially account for the GC language of each dummy except for the unaccessed dummy. E XORs on the encoded language for the accessed element and the encoded language for the unaccessed dummy. This allows E to solder the RAM output to the GC such that computation can continue.
5. Parties write back an encoding either of the just-accessed-element (for a read) or of the written element (for a write). This element is written to the smallest level. Parties also write a fresh dummy to the smallest level.
6. G applies a permutation to appropriate RAM levels.
7. After the n th access, E flushes the RAM by reading each index without writing anything back, then initializes a new RAM with the flushed values.

We formalize our GRAM in Sect. 5.4.

3 Related Work

Garbled RAM. [LO13] were the first to achieve sublinear random access in GC. As already mentioned, their GRAM evaluates a PRF inside the GC and also requires a circular-security assumption.

This circularity opened the door to further improvements. [GHL+14] gave two constructions, one that assumes identity-based-encryption and a second that assumes only one-way functions, but that incurs super-polylogarithmic overhead. [GLOS15] improved on this by constructing a GRAM that simultaneously assumes only one-way functions and that achieves polylog overhead. Both of these works avoid the [LO13] circularity assumption, but are expensive because they repeatedly evaluate cryptographic primitives inside the GC.

[GLO15] were the first to achieve a GRAM that makes only black-box use of crypto-primitives. Our lazy permutation network is inspired by [GLO15]: the authors describe a network of GCs, each of which can pass the program control flow to one of several other circuits. In this way they translate between GC languages. Our approach improves over the [GLO15] approach in several ways:

- The [GLO15] GRAM incurs factor κ blowup when passing messages through their network of GCs. Our lazy permutation network avoids this blowup.
- [GLO15] uses a costly probabilistic argument. Each node of their network is connected to a number of other nodes; this number scales with the statistical security parameter. The authors show that the necessary routing can be achieved at runtime with overwhelming probability.⁵ This approach uses a network that is *significantly* larger than is needed for any particular routing, and most nodes are ultimately wasted. In contrast, our lazy permutation network is direct. Each node connects to exactly two other nodes, and all connections are fully utilized over n accesses.
- [GLO15] compile their GRAM using off-the-shelf ORAM, incurring multiplicative overhead between their network of GCs and the ORAM. We build a custom RAM that makes minimal use of our lazy permutation network.

In this work, we focus on RAM access in the standard GC setting. A number of other works have explored other dimensions of GRAM, such as parallel RAM access, adaptivity, and succinctness [CCHR16, CH16, LO17, GOS18].

Practical GC and ORAM. Due to space, we defer discussion of works in the areas of practical GC and ORAM to the full version of this paper.

4 Preliminaries, Notation, and Assumptions

4.1 Common Notation

- G is the circuit generator. We refer to G as he/him.
- E is the circuit evaluator. We refer to E as she/her.
- We denote by $\langle x, y \rangle$ a pair of values where G holds x and E holds y .
- κ is the computational security parameter (e.g. 128).
- We write $x \triangleq y$ to denote that x is defined to be y .
- $\stackrel{c}{=}$ is the computational indistinguishability relation.
- $x \leftarrow y$ denotes that variable x is assigned to value y ; x can later be reassigned.
- We generally use n to denote the number of elements and w to denote the bit-width of those elements.
- $[x]$ denotes the natural numbers $0, \dots, x - 1$.

Our construction is a garbling scheme [BHR12], not a protocol. I.e., our construction is merely a tuple of procedures that can be plugged into GC protocols. However, it is often easier to think of G and E as participating in a semi-honest protocol. Thus, we often write that the parties “send messages”. We make two notes about this phrasing:

- We will never write that E sends a message to G : all information flows from G to E . In this way, we preserve the constant round nature of GC.
- ‘ G sends x to E ’ formally means that (1) our garbling procedure appends x to the GC and (2) our evaluation procedure extracts x from the GC.

⁵ The [GLO15] probabilistic argument *requires* that indices be accessed randomly. I.e., the [GLO15] leaky array cannot be used except by plugging it into ORAM.

4.2 Cryptographic Assumptions

We use the Free XOR technique [KS08], so we assume a circular correlation robust hash function H [CKKZ12, ZRE15]. In practice, we instantiate H using fixed-key AES [GKWY20].

4.3 Garbling Schemes

A *garbling scheme* [BHR12] is a method for securely computing a class of circuits in constant rounds. A garbling scheme is *not* a protocol; rather, it is a tuple of procedures that can be plugged into a variety of protocols.

Definition 1 (Garbling Scheme). A garbling scheme for a class of circuits $\mathbb{C}$ is a tuple of procedures:

$$(Gb, En, Ev, De)$$

where (1) Gb maps a circuit $C \in \mathbb{C}$ to a garbled circuit $\tilde{C}$, an input encoding string e , and an output decoding string d ; (2) En maps an input encoding string e and a cleartext bitstring x to an encoded input; (3) Ev maps a circuit C , a garbled circuit $\tilde{C}$, and an encoded input to an encoded output; and (4) De maps an output decoding string d and encoded output to a cleartext output string.

A garbling scheme must be *correct* and may satisfy any combination of *obliviousness*, *privacy*, and *authenticity* [BHR12]. We include formal definitions of these properties in the full version of this paper. Our scheme satisfies each definition and hence can be plugged into GC protocols.

4.4 Garblings and Sharings

We work with two kinds of encodings of logical values: ‘garblings’ and simple XOR shares. Garblings correspond to the traditional notion of garbled labels; i.e., a garbling is a length- κ value held by each party.

Recall from Sect. 2 that we manipulate languages inside the GC. This is why we work also with simple XOR sharings: we use XOR sharings to encode and move languages inside the GC. We define notation for both types of shares, and we emphasize the compatibility of garblings and sharings.

Garblings are Free XOR-style garbled circuit labels [KS08]. G samples a uniform value $\Delta \in \{0, 1\}^{\kappa-1}$. I.e., Δ is uniform except that the least significant bit is one. Δ is *global* to the entire computation. A garbling of $x \in \{0, 1\}$ is a tuple $\langle X, X \oplus x\Delta \rangle$, where the first element (here, X) is held by G , and the second by E .

Definition 2 (Garbling). Let $x \in \{0, 1\}$ be a bit. Let $X \in \{0, 1\}^\kappa$ be a bitstring held by G . We say that the pair $\langle X, X \oplus x\Delta \rangle$ is a garbling of x over (usually implicit) $\Delta \in \{0, 1\}^{\kappa-1}$. We denote a garbling of x by writing $\llbracket x \rrbracket$:

$$\llbracket x \rrbracket \triangleq \langle X, X \oplus x\Delta \rangle$$

Definition 3 (Sharing). Let $x, X \in \{0, 1\}$ be two bits. We say that the pair $\langle X, X \oplus x \rangle$ is a sharing of x . We denote a sharing of x by writing $\llbracket x \rrbracket$:

$$\llbracket x \rrbracket \triangleq \langle X, X \oplus x \rangle$$

We refer to G 's share X as the *language* of the garbling (resp. sharing). Except in specific circumstances, we use uniformly random languages both for garblings and for sharings.

Note, XOR is homomorphic over garblings [KS08] and sharings:

$$\{\!\{a\}\!\} \oplus \{\!\{b\}\!\} = \{\!\{a \oplus b\}\!\} \quad \llbracket a \rrbracket \oplus \llbracket b \rrbracket = \llbracket a \oplus b \rrbracket$$

We extend our garbling and sharing notation to vectors of values. That is, a garbling (resp. sharing) of a vector is a vector of garblings (resp. sharings):

$$\{\!\{a_0, \dots, a_{n-1}\}\!\} \triangleq (\{\!\{a_0\}\!\}, \dots, \{\!\{a_{n-1}\}\!\}) \quad \llbracket a_0, \dots, a_{n-1} \rrbracket \triangleq (\llbracket a_0 \rrbracket, \dots, \llbracket a_{n-1} \rrbracket)$$

Remark 3 (Length of garblings/sharings). Garblings are longer than sharings. I.e., let $x \in \{0, 1\}$ be a bit. Then $\{\!\{x\}\!\}$ is a pair of length- κ strings held by G and E . Meanwhile, $\llbracket x \rrbracket$ is a pair of bits held by G and E .

Remark 4 (Sharings contain garblings). Notice that the space of sharings contains the space of garblings. Indeed, this will be important later: we will in certain instances reinterpret a garbling $\{\!\{x\}\!\}$ as a sharing $\llbracket x\Delta \rrbracket$. This will allow us to operate on the garbling as if it is a sharing.

We frequently deal with values that are known to a particular party. We write x^G (resp. x^E) to denote that x is a value known to G (resp. to E) in cleartext. E.g., $\{\!\{x^E\}\!\}$ indicates a garbling of x where E knows x .

Operations on Sharings/Garblings.

- $\{\!\{x\}\!\} \mapsto \llbracket x \rrbracket$. Recall that G ensures that the least significant bit of Δ is one. Suppose each party takes the least significant bit of his/her part of $\{\!\{x\}\!\}$:

$$\begin{aligned} \text{lsb}(\{\!\{x\}\!\}) &= \text{lsb}(\langle X, X \oplus x\Delta \rangle) \triangleq \langle \text{lsb}(X), \text{lsb}(X \oplus x\Delta) \rangle \\ &= \langle \text{lsb}(X), \text{lsb}(X) \oplus x \cdot \text{lsb}(\Delta) \rangle = \langle \text{lsb}(X), \text{lsb}(X) \oplus x \rangle = \llbracket x \rrbracket \end{aligned}$$

That is, if both parties compute *lsb* on their parts of a garbling, the result is a valid sharing of the garbled value. This idea was first used to implement the classic point and permute technique.

- $\llbracket x \rrbracket \mapsto x^E$ and $\{\!\{x\}\!\} \mapsto x^E$. G can open the cleartext value of a sharing by sending his share to E . Similarly, we can open a garbling by first computing *lsb* (see above) and then opening the resulting share.
- $x^G \mapsto \llbracket x \rrbracket$ and $x^G \mapsto \{\!\{x\}\!\}$. G can easily introduce fresh inputs. Specifically, let x be a bit chosen by G and unknown to E . The parties can construct $\langle x, 0 \rangle = \llbracket x \rrbracket$. Similarly, the parties can construct $\langle x\Delta, 0 \rangle = \{\!\{x\}\!\}$.
- $\{\!\{x\}\!\} \cdot \{\!\{y\}\!\} \mapsto \{\!\{x \cdot y\}\!\}$. Garblings support AND gates. This operation can be implemented using two ciphertexts [ZRE15] (or 1.5 ciphertexts [RR21]).

- INPUT:
 - G inputs a permutation on n elements π .
 - A garbled array $\llbracket x_0, \dots, x_{n-1} \rrbracket$ where $x_i \in \{0, 1\}^w$.
- OUTPUT:
 - The permuted array $\llbracket \pi(x_0, \dots, x_{n-1}) \rrbracket$.

Fig. 2. Interface to the procedure G -permute which permutes n values using a permutation π chosen by G . For power of two n , permuting n garbled values each of length w costs $w \cdot (n \log n - n + 1) \cdot \kappa$ bits of communication via a permutation network [Wak68].

- $x^G \cdot \llbracket y \rrbracket \mapsto \llbracket x \cdot y \rrbracket$. It is possible to instantiate a cheaper AND gate if G knows in cleartext one of the arguments. This operation can be implemented using one ciphertext [ZRE15].
- $\llbracket x^E \rrbracket \cdot \llbracket y \rrbracket \mapsto \llbracket x \cdot y \rrbracket$. This novel operation scales a vector of sharings by a garbling whose cleartext value is known to E . Section 5.1 gives the procedure.
- $\llbracket x \rrbracket \cdot y^G \mapsto \llbracket x \cdot y \rrbracket$. This operation follows simply from the above scaling procedure. See Sect. 5.1.

4.5 Oblivious Permutation

We permute garbled arrays using permutations chosen by G . A permutation on $n = 2^k$ width- w elements can be implemented using $w(n \log n - n + 1)$ AND gates via a classic construction [Wak68]. Since G chooses the permutation, we can use single ciphertext AND gates and implement the permutation for only $w \cdot (n \log n - n + 1) \cdot \kappa$ bits. Figure 2 lists the interface to this procedure.

5 Approach

In this section we formalize the approach described in Sect. 2. Our formalism covers four topics:

- Section 5.1 formalizes our generalized GC gates. These gates allow us to avoid the factor- κ blowup that is common to prior GRAMs.
- Section 5.2 uses these new gates to modify an existing pop-only stack construction [ZE13]. Our modified pop-only stacks leak their access pattern to E but can efficiently store GC languages.
- Section 5.3 uses pop-only stacks to formalize our lazy permutation network.
- Section 5.4 builds on the lazy permutation network to formalize our GRAM.

We package the algorithms and definitions in this section into a garbling scheme [BHR12] that we call EPIGRAM. EPIGRAM handles arbitrary circuits with AND gates, XOR gates, and array accesses, and is defined as follows:

Construction 1. (EPIGRAM). EPIGRAM is a garbling scheme (Definition 1) that handles circuits with four kinds of gates:

- *XOR gates take as input two bits and output the XOR of the two inputs.*
- *AND gates take as input two bits and output the AND of the two inputs.*
- *ARRAY gates are parameterized over power of two n and positive integer w . The gate outputs a zero-initialized array of n elements each of width w .*
- *ACCESS gates take as input (1) an array A , (2) a $(\log n)$ -bit index α , (3) a w -bit value y to store in the case of a write, and (4) a bit r that indicates if this is a read or write. The gate outputs $A[\alpha]$. As a side effect, A is mutated:*

$$A[\alpha] \leftarrow \begin{cases} y & \text{if } r = 0 \\ A[\alpha] & \text{otherwise} \end{cases}$$

The garbling scheme procedures are defined as follows:

- *En and De are standard; formally, our scheme is projective [BHR12], which allows us to implement En and De as simple maps between cleartext and encoded values. We formalize En and De in the full version of this paper.*
- *Ev and Gb each proceed gate-by-gate through the circuit. For each XOR gate, each procedure XORs the inputs [KS08]. For each AND gate, the procedures compute the half-gates approach [ZRE15]. For each ARRAY gate, Gb (resp. Ev) invokes G's (resp. E's) part of the array initialization procedure (Fig. 9). For each ACCESS gate, Gb (resp. Ev) invokes G's (resp. E's) part of the array access procedure (Fig. 10).*

In the full version of this paper, we prove lemmas and theorems that together imply the following result:

Theorem 1 (Main Theorem). *If H is a circular correlation robust hash function, then EPIGRAM is a correct, oblivious, private, and authentic garbling scheme. For each ACCESS gate applied to an array of n elements each of size $w = \Omega(\log^2 n)$, Gb outputs a GC of amortized size $O(w \cdot \log^2 n \cdot \kappa)$ and both Gb and Ev consume amortized $O(w \cdot \log^2 n \cdot \kappa)$ computation.*

5.1 Avoiding Factor κ Blowup

Recall from Sect. 2 that we avoid the factor- κ overhead that is typical in GRAMs. We now give the crucial operation that enables this improvement.

Our operation scales a vector of κ sharings by a garbled bit whose value is known to E . The scaled vector remains hidden from E . The operation computes $\{x^E\} \cdot \{y\} \mapsto \{x \cdot y\}$ for $y \in \{0, 1\}^\kappa$ (see Fig. 3). Crucially, the operation only requires that G send to E κ total bits. While this presentation is novel, the procedure in Fig. 3 is a simple generalization of techniques given in [ZRE15]. This generalization allows us to scale an encoded GC language of length w (when $w = c \cdot \kappa$ for some c) for only w bits. This is how we avoid factor- κ blowup.

Formally, we have a *vector space* where the vectors are sharings and the scalars are garblings whose value is known to E . Vector space operations cannot compute arbitrary functions of sharings, but they can arbitrarily move sharings around. These data movements suffice to build our lazy permutation network.

Given Fig. 3, we can also compute $\{x\} \cdot y^G \mapsto \{x \cdot y\}$ for $y \in \{0, 1\}^\kappa$:

– INPUT:

- A garbled bit known to E : $\{x^E\}$.
- A shared vector $\llbracket y \rrbracket$ for $y \in \{0, 1\}^\kappa$.

– OUTPUT:

- A sharing of the scaled vector $\llbracket x \cdot y \rrbracket$.

– PROCEDURE $\{x^E\} \cdot \llbracket y \rrbracket$:

- Parties agree on a gate-specific nonce i .
- Let $\langle X, X \oplus x\Delta \rangle = \{x^E\}$.
- Let $\langle Y, Y \oplus y \rangle = \llbracket y \rrbracket$.
- G computes and sends to E $row \triangleq H(X \oplus \Delta, i) \oplus H(X, i) \oplus Y$.
- E computes the following:

$$\begin{aligned}
 & H(X \oplus x\Delta, i) \oplus x \cdot (row \oplus (Y \oplus y)) \\
 = & H(X \oplus x\Delta, i) \oplus \begin{cases} row \oplus (Y \oplus y) & \text{if } x = 1 \\ 0 & \text{otherwise} \end{cases} \\
 = & \begin{cases} H(X \oplus \Delta, i) \oplus (H(X \oplus \Delta, i) \oplus H(X, i) \oplus Y) \oplus Y \oplus y & \text{if } x = 1 \\ H(X, i) & \text{otherwise} \end{cases} \\
 = & H(X, i) \oplus x \cdot y
 \end{aligned}$$

- Parties output (respective shares of) $\langle H(X, i), H(X, i) \oplus x \cdot y \rangle = \llbracket x \cdot y \rrbracket$.

Fig. 3. Scaling a shared κ -bit vector by a garbling where E knows in cleartext the scalar. Scaling a κ -bit sharing requires that G send to E κ bits. We prove the construction secure when G 's share of the vector $\llbracket y \rrbracket$ is either (1) a uniform bitstring Y or (2) a bitstring $z\Delta$ for $z \in \{0, 1\}$. The latter case arises when G introduces a garbled input.

– PROCEDURE $\{x\} \cdot y^G$:

- Parties compute $\llbracket x \rrbracket = \text{lsb}(\{x\})$. Let $\langle X, X \oplus x \rangle = \{x\}$.
- G introduces inputs $\{X\}$, $\llbracket y \rrbracket$ and $\llbracket X \cdot y \rrbracket$.
- Parties compute $\{X\} \oplus \{x\} = \{X \oplus x\}$. Note that E knows $X \oplus x$.
- Parties compute (using Fig. 3) and output:

$$\{X \oplus x\} \cdot \llbracket y \rrbracket \oplus \llbracket X \cdot y \rrbracket = \llbracket (X \oplus x) \cdot y \rrbracket \oplus \llbracket X \cdot y \rrbracket = \llbracket x \cdot y \rrbracket$$

This procedure is useful in our lazy permutation network and in the $\mathcal{C}_{hide}$ circuit.

5.2 Pop-only Oblivious Stacks

Our lazy permutation network uses pop-only oblivious stacks [ZE13], a data structure with a single pop operation controlled by a garbled bit. If the bit is one, then the stack indeed pops. Otherwise, the stack returns an encoded zero and is left unchanged. Typically, both the data stored in the stack *and* the access pattern are hidden. For our purposes, we only need a stack where the stored data is hidden from E , but where E learns the access pattern.

[ZE13] gave an efficient circuit-based stack construction that incurs only $O(\log n)$ overhead per pop. This construction stores the data across $O(\log n)$

– INPUT: • A block of n elements $\llbracket x_0, \dots, x_{n-1} \rrbracket$ where $x_i \in \{0, 1\}^w$. – OUTPUT: • A capacity n stack $Stack(x_0, \dots, x_{n-1})$.
– INPUT: • A size n stack $Stack(x_0, \dots, x_{n-1})$. • A garbled bit known to E $\llbracket p^E \rrbracket$ that indicates whether or not to pop. – OUTPUT: • The popped value $\llbracket p \cdot x_0 \rrbracket$. • The updated stack: $\begin{cases} Stack(x_1, \dots, x_{n-1}, 0^w) & \text{if } p = 1 \\ Stack(x_0, \dots, x_{n-1}) & \text{otherwise} \end{cases}$

Fig. 4. Interface to stack procedures *stack-init* (top) and *pop* (bottom). For a stack of size n with width- w entries, parties locally initialize using $O(w \cdot n)$ computation; each pop costs amortized $O(w \cdot \log n)$ communication and computation.

levels of exponentially increasing size; larger levels are touched exponentially less often than smaller levels, yielding low logarithmic overhead.

If E is allowed to learn the access pattern, we can implement the [ZE13] construction where the stack holds arbitrary sharings, not just garblings. This is done by replacing AND gates – which move data towards the top of the stack – with our scaling gate (Fig. 3). Since we simply replace AND gates by scaling gates, we do not further specify. A modified stack with n elements each of width w costs amortized $O(w \cdot \log n)$ bits of communication per pop.

Construction 2 (Pop-only Stack). *Let $x_0, \dots, x_{n-1}$ be a n elements such that $x_i \in \{0, 1\}^w$. $Stack(x_0, \dots, x_{n-1})$ is a pop-only stack of elements $x_0, \dots, x_{n-1}$. Pop-only stacks support the procedures *stack-init* and *pop* (Fig. 4).*

5.3 Lazy Permutations

Recall from Sect. 2 that our lazy permutation network allows E to look up an encoded physical address and an encoded language for the needed RAM slot. The network is a binary tree where each inner node holds two pop-only oblivious stacks. Each inner node forwards messages to its children. Once a message is forwarded all the way to a leaf, the leaf node interprets the message as (1) an encoding of the current RAM time and (2) an encoding of an output language. This leaf node accordingly computes encodings of the appropriate physical address and language, then translates these to the output language. The encoded address and language are later used to allow E to read from RAM.

Inner Nodes and Implementation of Garbled Switches. For simplicity of notation, let level 0 denote the tree level that holds the leaves; level $\log n$ holds

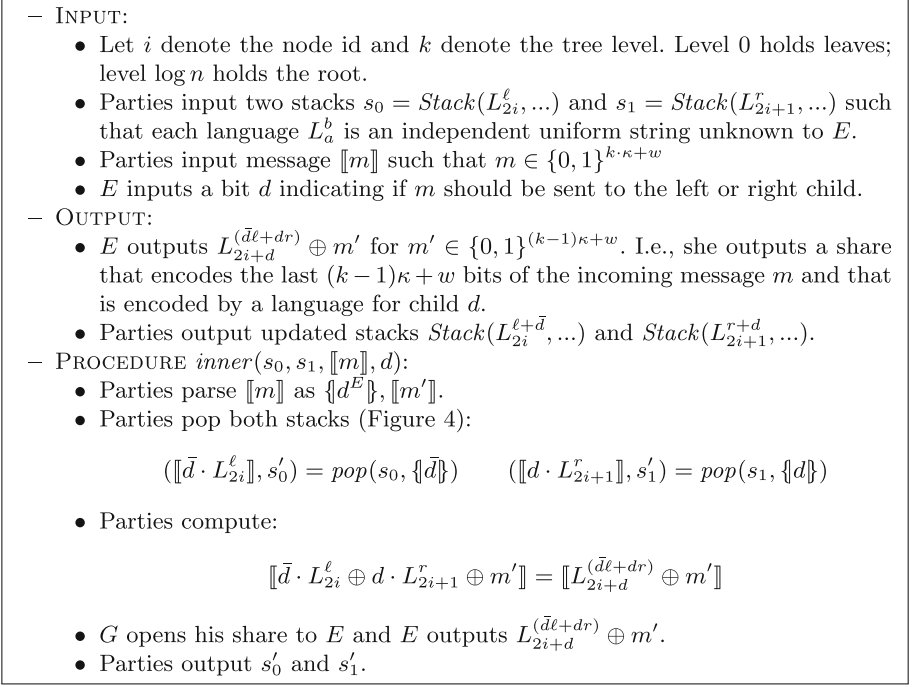


Fig. 5. Procedure for inner nodes of a lazy permutation network, implementing garbled switches.

the root. Consider an arbitrary inner node i on level k . This node can 2^k times receive a message $\llbracket m \rrbracket$ of a fixed, arbitrary length. On each message, the node strips the first κ bits from the message and interprets them as the garbling of a bit $\{d\}$. d is a direction indicator: if $d = 0$, then the node forwards the remaining message to its left child; otherwise it forwards to its right child. Over its lifetime, the inner node forwards 2^{k-1} messages to its left child and 2^{k-1} messages to its right child. Crucially, the *order* in which a node distributes its 2^k messages to its children is not decided until runtime.

Each of the 2^k messages are sharings with a particular language. I.e., the j th message $\llbracket m_j \rrbracket$ has form $\langle L_j, L_j \oplus m_j \rangle$ where each language L_j is distinct. The node must convert each message to a language next expected by the target child.

Assume that a particular node has so far forwarded ℓ messages to its left child and r messages to its right child. Let L_a^b denote the b th input language for node a . Note that the current language is thus $L_i^{\ell+r}$ and the language expected by the left (resp. right) child is L_{2i}^ℓ (resp. L_{2i+1}^r).

To forward m_j based on d , the node computes the following translation value:

$$\llbracket \bar{d} \cdot L_{2i}^\ell \oplus d \cdot L_{2i+1}^r \rrbracket = \llbracket L_{2i+d}^{(\bar{d}\ell+dr)} \rrbracket \quad (1)$$

- INPUT:
 - Let this node be leaf $\pi(p)$ where π is a permutation chosen by G .
 - G inputs the storage metadata (Definition 4) $\mathcal{M}_p$ for one-time index p .
 - Parties input $\{\!\{T\}\!\}$, a garbling of the current RAM time.
 - Parties input $\llbracket Y \rrbracket$, a sharing of an output language such that Y is uniform.
- OUTPUT:
 - Let $(t_i^p, @_i^p, X_i^p)_{i \in [\log n]} = \mathcal{M}_p$. Let t_j^p be the largest metadata timer such that $t_j^p \leq T$. E outputs $Y \oplus (@_j^p \cdot \Delta, X_j^p)$. I.e., she outputs a sharing of the appropriate physical address and language for one-time index p .
- PROCEDURE $leaf(\mathcal{M}_p, \{\!\{T\}\!\}, \llbracket Y \rrbracket)$:
 - Parties set $\{\!\{@\}\!\} \leftarrow \{\!\{@_0^p\}\!\}$ and $\llbracket X \rrbracket \leftarrow \llbracket X_0^p \rrbracket$.
 - For each $i \in \{1.. \log n - 1\}$ parties compute $\{\!\{t_i^p \leq T\}\!\}$ via a Boolean circuit.
 - For each $i \in \{1.. \log n - 1\}$ the parties update $\llbracket X \rrbracket$:

$$\begin{aligned}
 \llbracket X \rrbracket &\leftarrow \llbracket X \rrbracket \oplus \{\!\{t_i^p \leq T\}\!\} \cdot (X_{i-1}^p \oplus X_i^p) \\
 &= \llbracket X \oplus (t_i^p \leq T) \cdot (X_{i-1}^p \oplus X_i^p) \rrbracket && G \text{ knows } (X_{i-1}^p \oplus X_i^p) \\
 &= \begin{cases} \llbracket X \oplus X \oplus X_i^p \rrbracket & \text{if } t_i^p \leq T \\ \llbracket X \rrbracket & \text{otherwise} \end{cases} && (t_i^p \leq T) \Rightarrow (t_{i-1}^p \leq T) \text{ (Defn. 4)} \\
 &= \begin{cases} \llbracket X_i^p \rrbracket & \text{if } t_i^p \leq T \\ \llbracket X \rrbracket & \text{otherwise} \end{cases}
 \end{aligned}$$

We elaborate the above step carefully to show this conditional update can be achieved using efficient sharing procedures given in Section 5.1.

- For each $i \in \{1.. \log n - 1\}$ the parties update $\{\!\{@\}\!\}$ via a Boolean circuit:

$$\{\!\{@\}\!\} \leftarrow \begin{cases} \{\!\{@_i^p\}\!\} & \text{if } t_i^p \leq T \\ \{\!\{@\}\!\} & \text{otherwise} \end{cases}$$

- Let $\llbracket m \rrbracket \triangleq \{\!\{@\}\!\}, \llbracket X \rrbracket$ be the concatenated output. Then parties compute $\llbracket m \oplus Y \rrbracket$ and G opens his share to E .
- E outputs $m \oplus Y = Y \oplus (@_j^p \cdot \Delta, X_j^p)$.

Fig. 6. Procedure for leaf nodes of a lazy permutation network.

To compute the above, node i maintains two oblivious pop-only stacks (see Sect. 5.2) of size 2^{k-1} . The first stack stores, in order, sharings of the 2^{k-1} languages for the left child. The second stack similarly stores languages for the right child. By popping both stacks based on $\{\!\{d\}\!\}$, the node computes Eq. (1). Figure 5 specifies the formal procedure for inner nodes.

Leaf Nodes. Once a message has propagated from the root node to a leaf, we are ready to complete a lookup. Each leaf node of the lazy permutation network is a static circuit that outputs the encoding of a physical address and a language.

As the parties access RAM, G repeatedly permutes the physical storage to hide the access pattern from E . Each one-time index p has $O(\log n)$ different

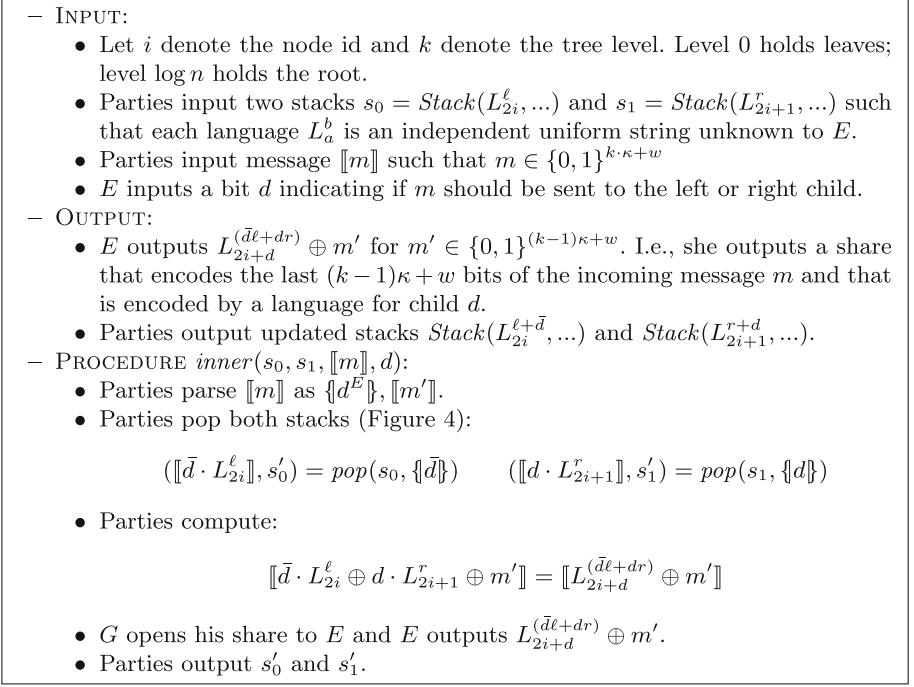


Fig. 7. Lazy permutation network initialization. When initializing with leaves that store languages of length w , G sends to E a GC of size $O(w \cdot n \cdot \log^2 n)$ bits.

physical addresses and languages; the needed address and language depends on how many accesses have occurred. Thus, each leaf node must conditionally output one of $O(\log n)$ values depending on how many accesses have occurred.

G chooses all permutations and storage languages before the first RAM access. Hence, G can precompute metadata indicating which one-time index will be stored where and with what language at which point in time:

Definition 4 (Storage Metadata). Consider a one-time index p . The storage metadata $\mathcal{M}_p$ for one-time index p is a sequence of $\log n$ three-tuples:

$$\mathcal{M}_p \triangleq (t_i^p, @_i^p, L_i^p)_{[i \in \log n]}$$

where each t_i^p is a natural number that indicates a point in time, $@_i^p$ is a physical address, and L_i^p is a uniform language. Each time $t_i \leq t_{i+1}$.

In our construction, each one-time index p may have fewer than $\log n$ corresponding physical addresses. G pads storage metadata by repeating the last entry until all $\log n$ slots are filled. G uses the storage metadata for each one-time index to configure each leaf. Figure 6 specifies the procedure for leaf nodes.

- INPUT:
 - A size n lazy permutation network $\tilde{\pi}$.
 - A garbled index $\{\pi(p)\}$ such that $\pi(p)$ has not yet been routed.
 - The current RAM time T .
- OUTPUT:
 - A physical address $\{\textcircled{p}\}$.
 - A shared language $\llbracket X^p \rrbracket$.
 - The updated lazy permutation network (i.e., where $\pi(p)$ has been routed).
- PROCEDURE $route(\tilde{\pi}, \{\pi(p)\}, T)$:
 - Let v denote the number of times $\tilde{\pi}$ has already been used.
 - G and E parse the input lazy permutation network:

$$\left(\llbracket L_0^{j \in [n]} \rrbracket, (s_{i \in [n-1]}^\ell, s_{i \in [n-1]}^r) \right) = \tilde{\pi}$$

- G samples a uniform value Y with length appropriate for the output; the parties trivially hold $\llbracket Y \rrbracket$. The parties also hold $\llbracket L_0^v \rrbracket$.
- Parties collect $\llbracket m \rrbracket \triangleq \{\pi(p)\}, \{T\}, \llbracket Y \rrbracket$ and then compute $\llbracket L_0^v \oplus m \rrbracket$; G opens his share to E such that E holds $L_0^v \oplus m$.
- Recall from Figure 7 that at initialization, E stored 2^k GCs for each level k node. Let E initialize $M \leftarrow L_0^v \oplus m$. E now traverses the tree from root to leaf $\pi(p)$. At each node i on the path to $\pi(p)$, G invokes:

$$(M, s_j^\ell, s_j^r) \leftarrow inner(s_j^\ell, s_j^r, M, d)$$

where j is the id of the i th node on the path to $\pi(p)$ and d is the i th bit of $\pi(p)$. To perform each invocation, E loads in the j th GC stored at initialization. This propagates E 's share of $\{T\}$ and $\llbracket Y \rrbracket$ to leaf $\pi(p)$.

- E invokes (using the appropriate GC) the leaf node procedure:

$$Y \oplus (\textcircled{p} \cdot \Delta, X^p) \leftarrow leaf(\cdot, \{T\}, \llbracket Y \rrbracket)$$

- The parties output the updated $\tilde{\pi}$.
- The parties compute and output:

$$\langle Y, Y \oplus (\textcircled{p} \cdot \Delta, X^p) \rangle = \llbracket \textcircled{p} \cdot \Delta, X^p \rrbracket = \{\textcircled{p}\}, \llbracket X^p \rrbracket$$

Fig. 8. Procedure to route one value through a lazy permutation network.

Putting the Network Together. We now formalize the top level lazy permutation network. To instantiate a new network, G and E agree on a size n and a width w and G provides storage metadata, conveying the information that should be stored at the leaves of the network. From here, G proceeds node-by-node through the binary tree, fully garbling each node. E receives all such GCs from G , but crucially she does not yet begin to evaluate. Instead, she stores the GCs for later use, remembering which GCs belong to each individual node.

Recall that G selects a uniform permutation π that prevents E from viewing the one-time index access pattern: when the GC requests access to one-time index p , E is shown $\pi(p)$. Now, let us consider the i th access to the network. At the time of this access, a garbled index $\{\pi(p)\}$ is given as input by the parties.

G selects a uniform language Y to use as the output language, and the parties trivially construct the sharing $\llbracket Y \rrbracket$. The parties then concatenate the message $\llbracket m_i \rrbracket \triangleq \{\pi(p)\}, \{T\}, \llbracket Y \rrbracket$ where T is the number of RAM writes performed so far. Let L_0^i denote the i th input language for the root node 0. The parties compute $\llbracket L_0^i \rrbracket \oplus \llbracket m_i \rrbracket$ and G sends his resulting share, giving to E a valid share of m_i with language configured for the root node. E now feeds this value into the tree, starting from the root node and traversing the path to leaf $\pi(p)$. Note that G does not perform this traversal, since he already garbled all circuits.

Each inner node strips off one garbled bit of $\pi(p)$. This propagates the message to leaf $\pi(p)$. Finally, the leaf node computes the appropriate physical address and language for one-time index p and translates them to language Y . Let $Y \oplus (@^p \cdot \Delta, L^p)$ denote E 's output from the leaf node. The parties output:

$$\langle Y, Y \oplus (@^p \cdot \Delta, L^p) \rangle = \llbracket @^p \cdot \Delta, L^p \rrbracket = \{\@^p\}, \llbracket L^p \rrbracket$$

Thus, the parties successfully read an address and a language from the network.

Construction 3. (Lazy Permutation Network). *Let n be a power of two. A size- n lazy permutation network $\tilde{\pi}$ is a two-tuple consisting of:*

1. *Sharings of the input languages to the root node $\llbracket L_0^{j \in [n]} \rrbracket$.*
2. *$2n - 2$ stacks belonging to the $n - 1$ inner nodes, $s_{i \in [n-1]}^l$ and $s_{i \in [n-1]}^r$.*

Here, each input language $L_0^{j \in [n]}$ and each language stored in each stack is an independently sampled uniform string. Lazy permutation networks support initialization (Fig. 7) and routing of a single input (Fig. 8).

5.4 Our GRAM

We formalize our GRAM on top of our lazy permutation network:

Construction 4. (GRAM). *Let n – the RAM size – be a power of two and let w – the word size – be a positive integer. Let $x_0, \dots, x_{n-1}$ be n values such that $x_i \in \{0, 1\}^w$. Then $\text{Array}(x_{i \in [n]})$ denotes a size- n GRAM holding the content $x_{i \in [n]}$. Concretely, a GRAM is a tuple consisting of:*

1. *A timer T denoting the number of writes performed so far.*
2. *A sequence of languages $\mathcal{X}$ held by G and used as the languages for the permuted RAM content. Each language has length $w \cdot \kappa$, sufficient to encode a single garbled word.*
3. *A size- $2n$ uniform permutation π held by G .*
4. *A sequence of $n + 1$ uniform permutations $\pi_0, \dots, \pi_n$ held by G and used to permute the physical storage. These hide the RAM access pattern from E .*
5. *A size- $2n$ lazy permutation $\tilde{\pi}$.*
6. *A recursively instantiated RAM called the index map that maps each logical index α to $\pi(p)$: the (permuted) one-time index where α is currently saved. For each recursive RAM of size n , we instantiate the index map with word size $w = 2(\log n + 1)$. To bound the recursion, we use a linear-scan based RAM when instantiating a index map that stores only $O(w \cdot \log^2 n)$ bits.*

- INPUT:
 - Let n denote a number of elements and let w denote the width of each element. The parties input a vector $\{x_0, \dots, x_{n-1}\}$ where $x_i \in \{0, 1\}^w$
- OUTPUT:
 - A length n random access array $Array(x_{i \in [n]})$.
- PROCEDURE $array-init(\{x_{i \in [n]}\})$:
 - Parties initialize the timer T to n , indicating the n initial writes.
 - G schedules all accesses and computes his needed metadata:

$$(\mathcal{X}, \mathcal{M}_{p \in [2n]}, \pi_{i \in [n+1]}) \leftarrow G\text{-schedule}(n, w)$$

- G uniformly samples a size- $2n$ permutation π .
- Parties instantiate the lazy permutation network: $\tilde{\pi} \leftarrow \tilde{\pi}\text{-init}(\pi, \mathcal{M}_{p \in [2n]})$
- G and E recursively initialize the index map with content $\{0, 1, \dots, n-1\}$, indicating that each index i starts in one-time index i :

$$index\text{-map} \leftarrow array\text{-init}(\{0, 1, \dots, n-1\})$$

- Parties zero initialize the stash and each of the $\log n + 2$ levels of storage.
- Parties store the initial data $\{x_{i \in [n]}\}$ on level $\log n - 1$.^a

^a This is a simple trick. On each access, we shuffle RAM levels (see Figure 10) By initializing the content on level $\log n - 1$, we ensure that the first access will shuffle the n items with n dummies and place them on level $\log n$.

Fig. 9. RAM initialize.

7. $\log n + 2$ levels of physical storage where level i is a garbling of size $w \cdot 2^{i+1}$. Each level i is either vacant or stores 2^i real elements and 2^i dummies. The physical storage is permuted according to permutations $\pi_0, \dots, \pi_n$.
8. A garbling of size $2w$ called the stash. Parties write back to the stash; on each access, items are immediately moved from the stash into a level of storage.

GRAMs support initialization (Fig. 9) and access (Fig. 10).

Our top level garbling scheme is defined with respect to this data structure; EPIGRAM makes explicit calls to $array\text{-init}$ (Fig. 9) and $access$ (Fig. 10).

We call attention to $G\text{-schedule}$, $shuffle$, $flush$, and $hide$:

- $G\text{-schedule}$ is a local procedure run by G where he plans ahead for the next n accesses. Specifically, G selects uniform permutations on storage, chooses uniform languages with which to store the RAM content, and computes the storage metadata $\mathcal{M}_p$ for each one-time index $p \in [2n]$. The full version of this paper gives the explicit interface to $G\text{-schedule}$.
- $shuffle$ describes how G permutes levels of storage. By doing so, we ensure that the revealed physical addresses give no information to E . $shuffle$ is a straightforward formalization of the permutation schedule given in Sect. 2.4 and is formalized in the full version of this paper.

– INPUT:

- A length n array $A = \text{Array}(x_0, \dots, x_{n-1})$.
- A garbled index $\llbracket \alpha \rrbracket$ such that $\alpha \in \{0, 1\}^{\log n}$.
- A garbled value $\llbracket y \rrbracket$ to store in the case of a write.
- A garbled bit $\llbracket r \rrbracket$ that indicates if this is a read; else this is a write.

– OUTPUT:

- The indexed value $\llbracket x_\alpha \rrbracket$.
- The updated array $\text{Array}(x_0, \dots, x_{\alpha-1}, (r \cdot x_\alpha \oplus \bar{r} \cdot y), x_{\alpha+1}, \dots, x_{n-1})$.

– PROCEDURE $\text{access}(A, \llbracket \alpha \rrbracket, \llbracket y \rrbracket, \llbracket r \rrbracket)$:

- Parties appropriately permute levels of storage: $A \leftarrow \text{shuffle}(A)$
- If $T = 2n$ then the parties reinitialize and try again, returning that result:

$$\text{access}(\text{array-init}(\text{flush}(A)), \llbracket \alpha \rrbracket, \llbracket y \rrbracket, \llbracket r \rrbracket)$$

Otherwise, the parties continue as follows:

- Parties recursively access the index map and update the one-time index for index α by writing back a garbling $\llbracket \pi(T) \rrbracket$ (G knows $\pi(T)$):

$$\llbracket \pi(p) \rrbracket \leftarrow \text{access}(\text{index-map}, \llbracket \alpha \rrbracket, \llbracket \pi(T) \rrbracket, \llbracket 0 \rrbracket)$$

- G opens his share of $\llbracket \pi(p) \rrbracket$ to reveal $\pi(p)$ to E .
- E uses $\tilde{\pi}$ to route time T to leaf $\pi(p)$. This returns the current physical address and language corresponding to p .

$$(\llbracket @ \rrbracket, \llbracket X \rrbracket) \leftarrow \text{route}(\tilde{\pi}, \llbracket \pi(p) \rrbracket, T)$$

- For each populated storage level i , G chooses a previously unaccessed dummy element with address $@'_i$ and language D_i .
- Let j denote the level that holds $@$. Parties compute (See the full version for details of *hide*):

$$(\llbracket @_i \rrbracket, \llbracket D_j \rrbracket) \leftarrow \text{hide}(@'_i, D_i, \llbracket @ \rrbracket)$$

I.e., *hide* computes one physical address per populated storage level.

- G reveals to E each physical address $@_i$ by sending his share.
- E reads each physical address and XORs the values together. I.e., E reads each dummy language $D_{i \neq j}$ and the desired element $X \oplus x_\alpha \Delta$. This yields:

$$\left(\bigoplus_{i \neq j} D_i \right) \oplus X \oplus x_\alpha \Delta$$

- Let $\langle L, L \oplus X \rangle = \llbracket X \rrbracket$ and $\langle L', L' \oplus D_j \rangle = \llbracket D_j \rrbracket$. Parties compute and output:

$$\begin{aligned} & \left\langle L \oplus L' \oplus \left(\bigoplus_i D_i \right), L \oplus X \oplus L' \oplus D_j \oplus \left(\bigoplus_{i \neq j} D_i \right) \oplus X \oplus x_\alpha \Delta \right\rangle \\ &= \left\langle L \oplus L' \oplus \left(\bigoplus_i D_i \right), L \oplus L' \oplus \left(\bigoplus_i D_i \right) \oplus x_\alpha \Delta \right\rangle = \llbracket x_\alpha \rrbracket \end{aligned}$$

- Parties compute $\llbracket r \cdot x_\alpha \oplus \bar{r} \cdot y \rrbracket$ and place their shares in the first slot of the stash. Parties place $\llbracket 0 \rrbracket$, a fresh dummy, in the second slot of the stash.
- Parties increment the timer: $T \leftarrow T + 1$.

Fig. 10. RAM access.

– INPUT:

- A length n array $A = \text{Array}(x_0, \dots, x_{n-1})$.

– OUTPUT:

- The flushed content $\llbracket x_0, \dots, x_{n-1} \rrbracket$.

– PROCEDURE $\text{flush}(A)$:

- Parties recursively flush the index map:
$$\llbracket \pi(p_0), \dots, \pi(p_{n-1}) \rrbracket \leftarrow \text{flush}(\text{index-map})$$
- For each $i \in [n]$ the parties route time T to leaf $\pi(p_i)$, returning the current physical address and language corresponding to p_i :
$$(\llbracket @_i \rrbracket, \llbracket X_i \rrbracket) \leftarrow \text{route}(\tilde{\pi}, \llbracket \pi(p_i) \rrbracket, T)$$

When flushing, each level $i \neq \log n + 1$ is vacant, so we need not use extra machinery to hide the accessed level: E knows each item is on level $\log n + 1$.
- G reveals to E each physical address $@_i$ by sending his share.
- E reads each address $@_i$, yielding $X_i \oplus x_i \Delta$.
- For each i , let $\langle L_i, L_i \oplus X_i \rangle = \llbracket X_i \rrbracket$. Parties compute and output:
$$\langle L_i, L_i \oplus X_i \oplus X_i \oplus x_i \Delta \rangle = \llbracket x_i \rrbracket$$

Fig. 11. flush is a helper procedure used to reset the array after n accesses. flush recovers the n array elements and places them into a contiguous block.

- After each n -th access, we invoke flush (Fig. 11) to reinitialize GRAM. We also mention that our proof of correctness defines correctness of the GRAM data structure with respect to flush : a GRAM is *valid* if we can flush and recover its content.
- On each access, hide picks a dummy on each storage level, the conveys to E (1) a physical address on each level of storage and (2) a sharing of the language of the unaccessed dummy. The precise procedure is formalized in the full version of this paper.

With these four helper procedures defined, we formalize GRAM initialization (Fig. 9) and GRAM access (Fig. 10). Initialization is straightforward, and GRAM access is a formalization of the high level procedure given in Sect. 2.4.

6 Evaluation

In this section, we analyze EPIGRAM’s performance. We leave implementation and low-level optimization as important future work.

To estimate cost, we implemented a program that modularly computes the communication cost of each of EPIGRAM’s subcomponents. E.g., a permutation network on n width- w elements uses $w \cdot (n \log n - n + 1)$ ciphertexts [Wak68].

Figure 12 fixes the word size w to 128. That is, each RAM slot stores 128 *garbled* bits. We plot the estimated communication cost as a function of n . For comparison, we also plot the cost of a linear scan; a linear scan on n elements of

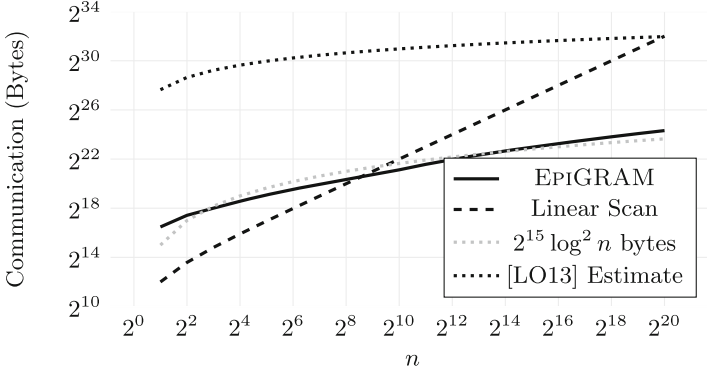


Fig. 12. Estimated concrete communication cost of our GRAM. We fix the word size $w = 128$ and plot per-access amortized communication as a function of n . For comparison we include an estimate of [LO13]’s performance (our estimate is favorable to [LO13], see the full version for our analysis).

width w and while using [ZRE15] AND gates can be achieved for (slightly more than) $2 \cdot w \cdot (n - 1)$ ciphertexts. We also plot the function $2^{15} \log^2 n$ bytes, a close approximation of EPIGRAM’s cost for $w = 128$.

Figure 12 clearly demonstrates EPIGRAM’s low polylogarithmic scaling. Note that our communication grows slightly faster than the function $2^{15} \log^2 n$. This can be explained by the fact that we *fixed* a relatively low and constant word size $w = 128$; recall that to achieve $O(\log^2 n)$ scaling, we must choose $w = \Omega(\log^2 n)$. Still, our cost is closely modeled by $O(\log^2 n)$.

EPIGRAM is practical even for small n . The breakeven point with trivial GRAM (i.e., GRAM implemented by linear scans) is only $n = 512$ elements. Even non-garbled ORAMs have similar breakeven points. For example, Circuit ORAM [WCS15] gives the breakeven point $w = 128, n = 128$. At $n = 2^{20}$, EPIGRAM consumes $\approx 200\times$ less communication than trivial GRAM.

Acknowledgements. This work was supported in part by NSF award #1909769, by a Facebook research award, a Cisco research award, and by Georgia Tech’s IISP cybersecurity seed funding (CSF) award. This material is also based upon work supported in part by DARPA under Contract No. HR001120C0087. Work of the third author is supported in part by DARPA under Cooperative Agreement HR0011-20-2-0025, NSF grant CNS-2001096, US-Israel BSF grant 2015782, Google Faculty Award, JP Morgan Faculty Award, IBM Faculty Research Award, Xerox Faculty Research Award, OKAWA Foundation Research Award, B. John Garrick Foundation Award, Teradata Research Award, Lockheed-Martin Research Award and Sunday Group. The views and conclusions contained herein are those of the authors and should not be interpreted as necessarily representing the official policies, either expressed or implied, of DARPA, the Department of Defense, or the U.S. Government. Distribution Statement “A” (Approved for Public Release, Distribution Unlimited). The U.S. Government is authorized to reproduce and distribute reprints for governmental purposes not withstanding any copyright annotation therein.

References

- [BHR12] Bellare, M., Hoang, V.T., Rogaway, P.: Foundations of garbled circuits. In: Yu, T., Danezis, G., Gligor, V.D. (eds.) ACM CCS 2012, pp. 784–796. ACM Press, October 2012
- [CCHR16] Canetti, R., Chen, Y., Holmgren, J., Raykova, M.: Adaptive succinct garbled RAM or: how to delegate your database. In: Hirt, M., Smith, A. (eds.) TCC 2016. LNCS, vol. 9986, pp. 61–90. Springer, Heidelberg (2016). https://doi.org/10.1007/978-3-662-53644-5_3
- [CH16] Canetti, R., Holmgren, J.: Fully succinct garbled RAM. In: Sudan, M., (ed.) ITCS 2016, pp. 169–178. ACM, January 2016
- [CKKZ12] Choi, S.G., Katz, J., Kumaresan, R., Zhou, H.-S.: On the security of the “Free-XOR” technique. In: Cramer, R. (ed.) TCC 2012. LNCS, vol. 7194, pp. 39–53. Springer, Heidelberg (2012). https://doi.org/10.1007/978-3-642-28914-9_3
- [GHL+14] Gentry, C., Halevi, S., Steve, L., Ostrovsky, R., Raykova, M., Wichs, D.: Garbled RAM revisited. In: Nguyen, P.Q., Oswald, E. (eds.) EUROCRYPT 2014. LNCS, vol. 8441, pp. 405–422. Springer, Heidelberg (2014)
- [GKWY20] Guo, C., Katz, J., Wang, X., Yu, Y.: Efficient and secure multiparty computation from fixed-key block ciphers. In: 2020 IEEE Symposium on Security and Privacy, pp. 825–841. IEEE Computer Society Press, May 2020
- [GLO15] Garg, S., Lu, S., Ostrovsky, R.: Black-box garbled RAM. In: Guruswami, V. (ed.) 56th FOCS, pp. 210–229. IEEE Computer Society Press, October 2015
- [GLOS15] Garg, S., Lu, S., Ostrovsky, R., Scafuro, A.: Garbled RAM from one-way functions. In: Servedio, R.A., Rubinfeld, R. (eds.) 47th ACM STOC, pp. 449–458. ACM Press, June 2015
- [GOS18] Garg, S., Ostrovsky, R., Srinivasan, A.: Adaptive garbled RAM from laconic oblivious transfer. Cryptology ePrint Archive, Report 2018/549 (2018). <https://eprint.iacr.org/2018/549>
- [HK21] Heath, D., Kolesnikov, V.: PrORAM: Fast $O(\log n)$ private coin ZK ORAM (2021)
- [KS08] Kolesnikov, V., Schneider, T.: Improved garbled circuit: free XOR gates and applications. In: Aceto, L., Damgård, I., Goldberg, L.A., Halldórsson, M.M., Ingólfssdóttir, A., Walukiewicz, I. (eds.) ICALP 2008. LNCS, vol. 5126, pp. 486–498. Springer, Heidelberg (2008). https://doi.org/10.1007/978-3-540-70583-3_40
- [LO13] Steve, L., Ostrovsky, R.: How to garble RAM programs. In: Johansson, T., Nguyen, P.Q. (eds.) EUROCRYPT 2013. LNCS, vol. 7881, pp. 719–734. Springer, Heidelberg (2013)
- [LO17] Steve, L., Ostrovsky, R.: Black-box parallel garbled RAM. In: Katz, J., Shacham, H. (eds.) CRYPTO 2017. Part II, volume 10402 of LNCS, pp. 66–92. Springer, Heidelberg (2017)
- [RR21] Rosulek, M., Roy, L.: Three halves make a whole? beating the half-gates lower bound for garbled circuits. In: Malkin, T., Peikert, C. (eds.) CRYPTO 2021. LNCS, vol. 12825, pp. 94–124. Springer, Cham (2021). https://doi.org/10.1007/978-3-030-84242-0_5
- [SvS+13] Stefanov, E., et al.: Path ORAM: an extremely simple oblivious RAM protocol. In: Sadeghi, A.-R., Gligor, V.D., Yung, M. (eds.) ACM CCS 2013, pp. 299–310. ACM Press, November 2013

- [Wak68] Waksman, A.: A permutation network. *J. ACM* **15**(1), 159–163 (1968)
- [WCS15] Xiao Wang, T.-H. Chan, H., Shi, E.: Circuit ORAM: On tightness of the Goldreich-Ostrovsky lower bound. In: Ray, I., Li, N., Kruegel, C. (eds.) *ACM CCS 2015*, pp. 850–861. ACM Press, October 2015
- [ZE13] Zahur, S., Evans, D.: Circuit structures for improving efficiency of security and privacy tools. In: *2013 IEEE Symposium on Security and Privacy*, pp. 493–507. IEEE Computer Society Press, May 2013
- [ZRE15] Zahur, S., Rosulek, M., Evans, D.: Two halves make a whole - reducing data transfer in garbled circuits using half gates. In: Oswald, E., Fischlin, M. (eds.) *EUROCRYPT 2015*. LNCS, vol. 9057, pp. 220–250. Springer, Heidelberg (2015). https://doi.org/10.1007/978-3-662-46803-6_8